

```
In [1]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [2]: #Task 1 – Spotify Songs Streamed per Day

df = pd.read_csv("spotify_daily_streams.csv")
```

```
In [3]: df
```

```
Out[3]:
```

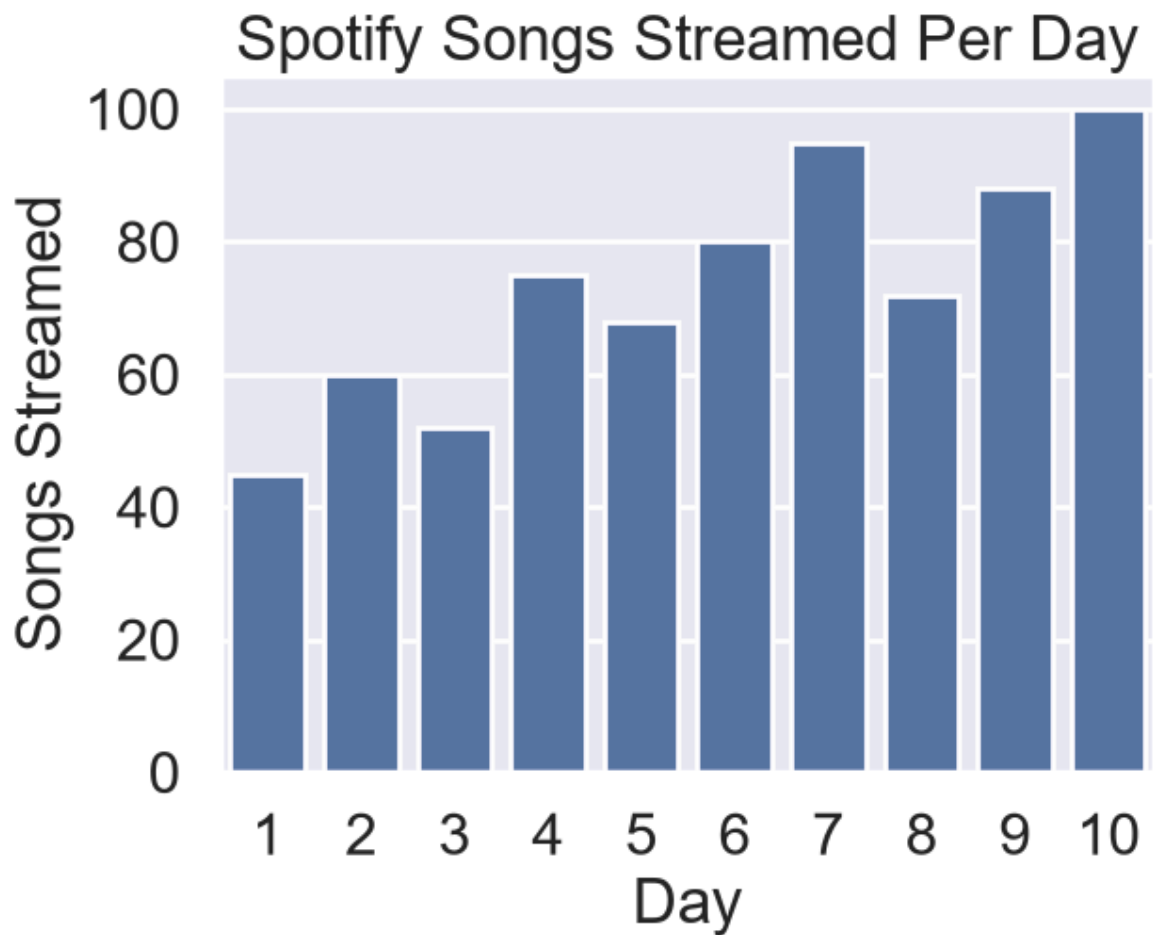
	day	songs_streamed
0	1	45
1	2	60
2	3	52
3	4	75
4	5	68
5	6	80
6	7	95
7	8	72
8	9	88
9	10	100

```
In [4]: sns.set(context="poster")
```

```
In [5]: sns.barplot( x="day", y="songs_streamed", data=df)

plt.xlabel("Day")
plt.ylabel("Songs Streamed")
plt.title("Spotify Songs Streamed Per Day")
```

```
Out[5]: Text(0.5, 1.0, 'Spotify Songs Streamed Per Day')
```



#Conclusion The poster context makes the plot elements, text, and labels larger than the default context.

In [6]: *#Task 2 – Food Orders by Cuisine*

```
df = pd.read_csv("food_orders_cuisine.csv")
```

In [7]: df

Out[7]:

	cuisine
0	Indian
1	Indian
2	Chinese
3	Italian
4	Indian
5	Chinese
6	Italian
7	Indian
8	Chinese
9	Indian
10	Italian
11	Chinese
12	Indian
13	Italian
14	Chinese
15	Indian
16	Chinese
17	Italian
18	Indian
19	Chinese

```
In [8]: sns.countplot(x="cuisine", data=df, palette="colorblind")

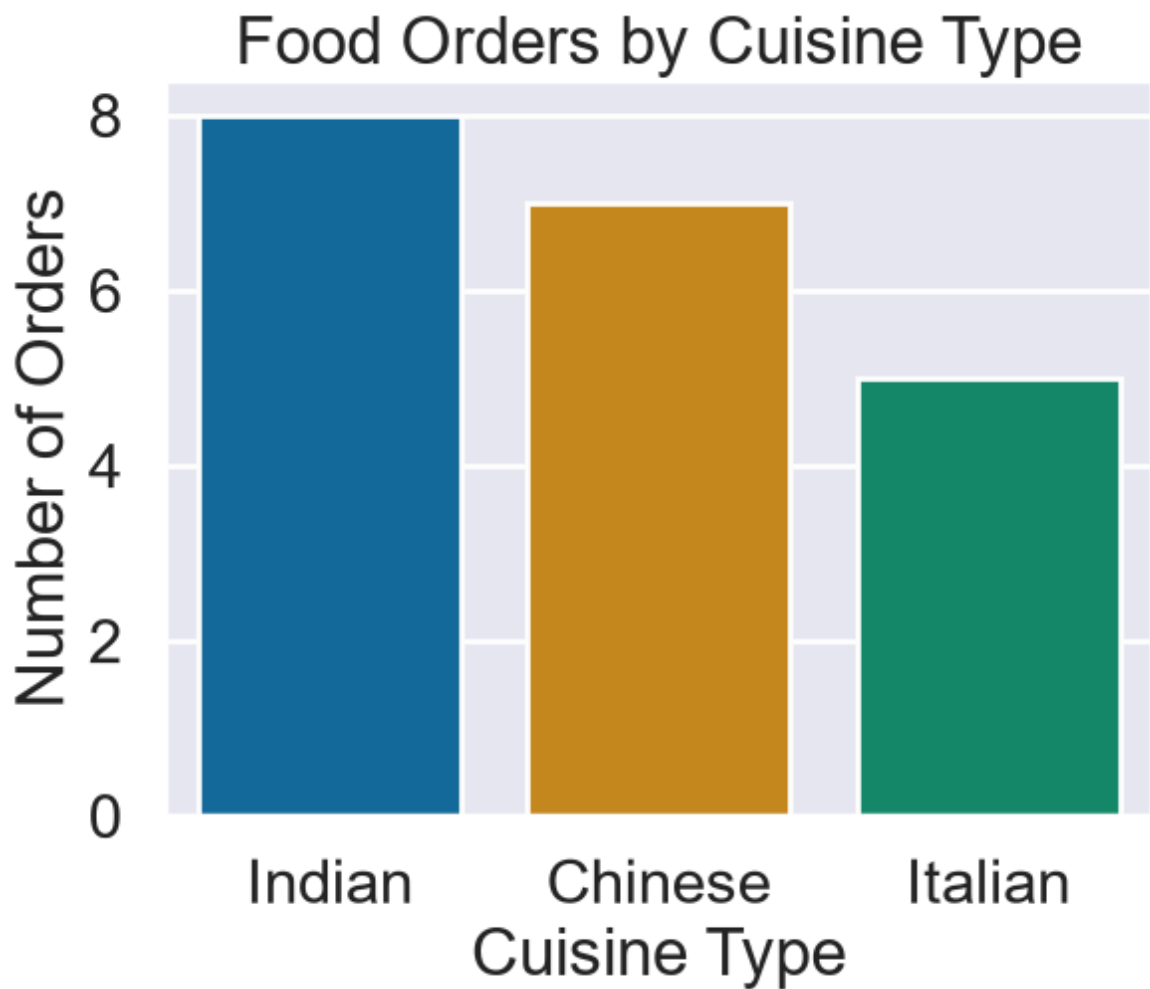
plt.xlabel("Cuisine Type")
plt.ylabel("Number of Orders")
plt.title("Food Orders by Cuisine Type")
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_8960\2440188013.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x="cuisine", data=df, palette="colorblind")
```

Out[8]: Text(0.5, 1.0, 'Food Orders by Cuisine Type')



#Conclusion- The countplot shows the number of food orders for each cuisine type. The colorblind palette provides colors that are easier for people with color-vision deficiencies to distinguish.

```
In [9]: # Task 3 – Daily Steps for 2 Weeks
```

```
df = pd.read_csv("daily_steps.csv")
```

```
In [10]: df
```

Out[10]:

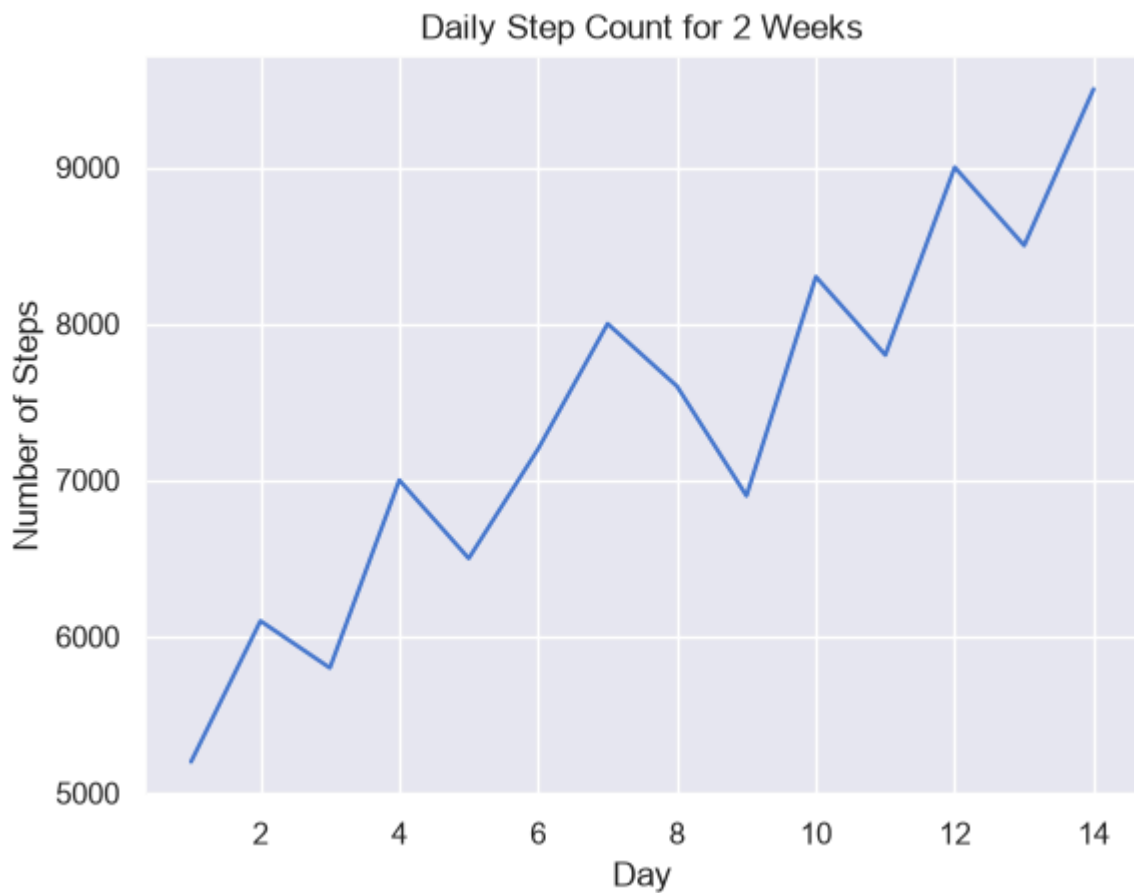
	day	steps
0	1	5200
1	2	6100
2	3	5800
3	4	7000
4	5	6500
5	6	7200
6	7	8000
7	8	7600
8	9	6900
9	10	8300
10	11	7800
11	12	9000
12	13	8500
13	14	9500

```
In [11]: sns.set(context="notebook", palette="muted")
```

```
In [12]: sns.lineplot(x="day", y="steps", data=df)

plt.xlabel("Day")
plt.ylabel("Number of Steps")
plt.title("Daily Step Count for 2 Weeks")
```

```
Out[12]: Text(0.5, 1.0, 'Daily Step Count for 2 Weeks')
```



#Conclusion The muted palette and notebook context give the lineplot a clean and simple appearance suitable for presentation.

In [13]: *#Task 4 – Change Deep to Colorblind and Talk to Poster*

```
df = pd.DataFrame({"category": ["A", "B", "C", "D", "E"], "sales": [50, 70, 45,
```

In [14]: df

Out[14]:

	category	sales
0	A	50
1	B	70
2	C	45
3	D	80
4	E	60

In [22]: `sns.set(context="talk", palette="deep")`

In [23]: `sns.barplot( x="category", y="sales", data=df)`

```
plt.title("Sales by Category")
```

Out[23]: Text(0.5, 1.0, 'Sales by Category')

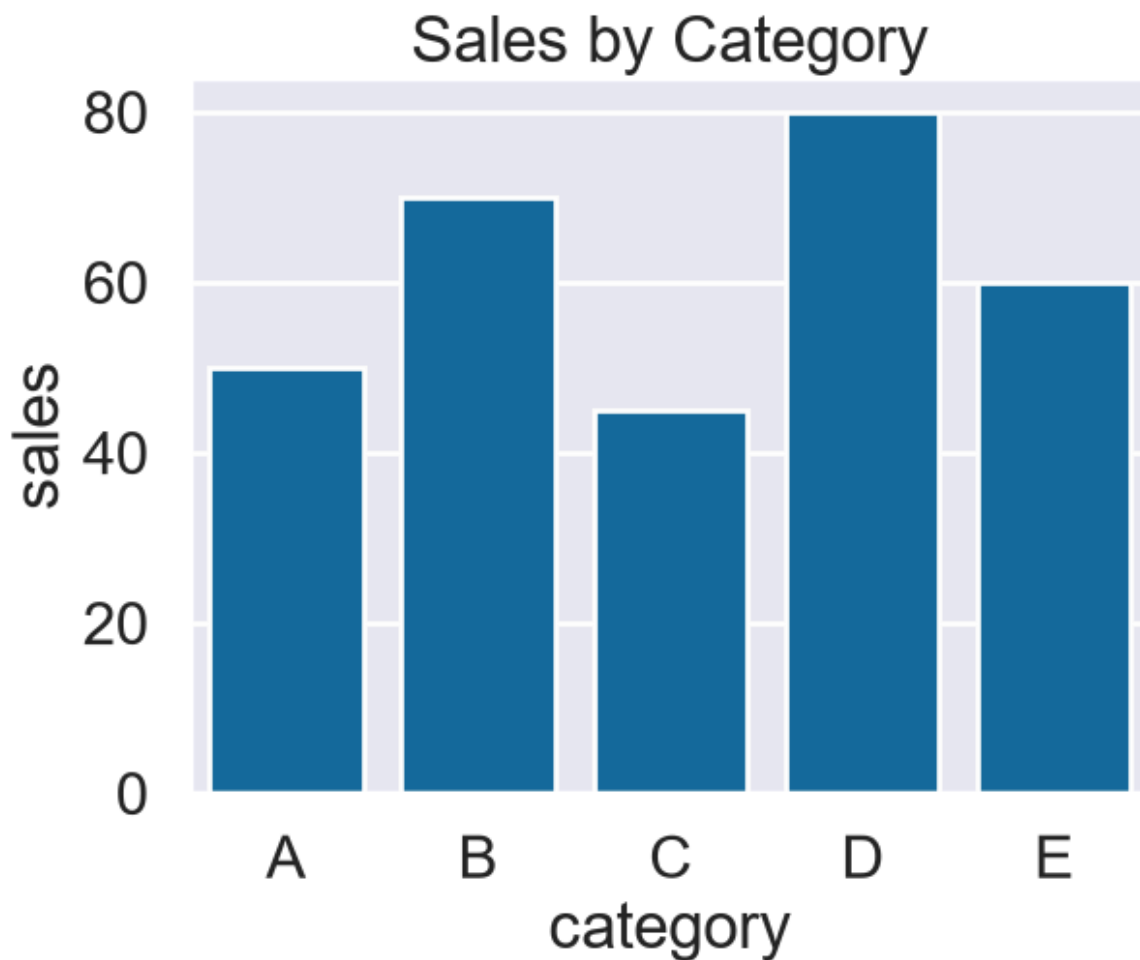


```
In [24]: sns.set(context="poster",palette="colorblind")
```

```
In [25]: sns.barplot( x="category", y="sales", data=df)

plt.title("Sales by Category")
```

```
Out[25]: Text(0.5, 1.0, 'Sales by Category')
```



#Conclusion Changing from talk to poster makes the plot elements larger. Changing from deep to colorblind provides a more accessible color palette.

In [26]: *#Task 5 – Flipkart Sales Dashboard Theme*

```
df = pd.read_csv("flipkart_sales.csv")
```

In [27]: df

Out[27]:

	category	sales
0	Mobiles	120000
1	Laptops	95000
2	TVs	70000
3	Headphones	55000
4	Accessories	40000

In [28]: `sns.set(context="talk", style="whitegrid", palette="deep", font_scale=1.0)`

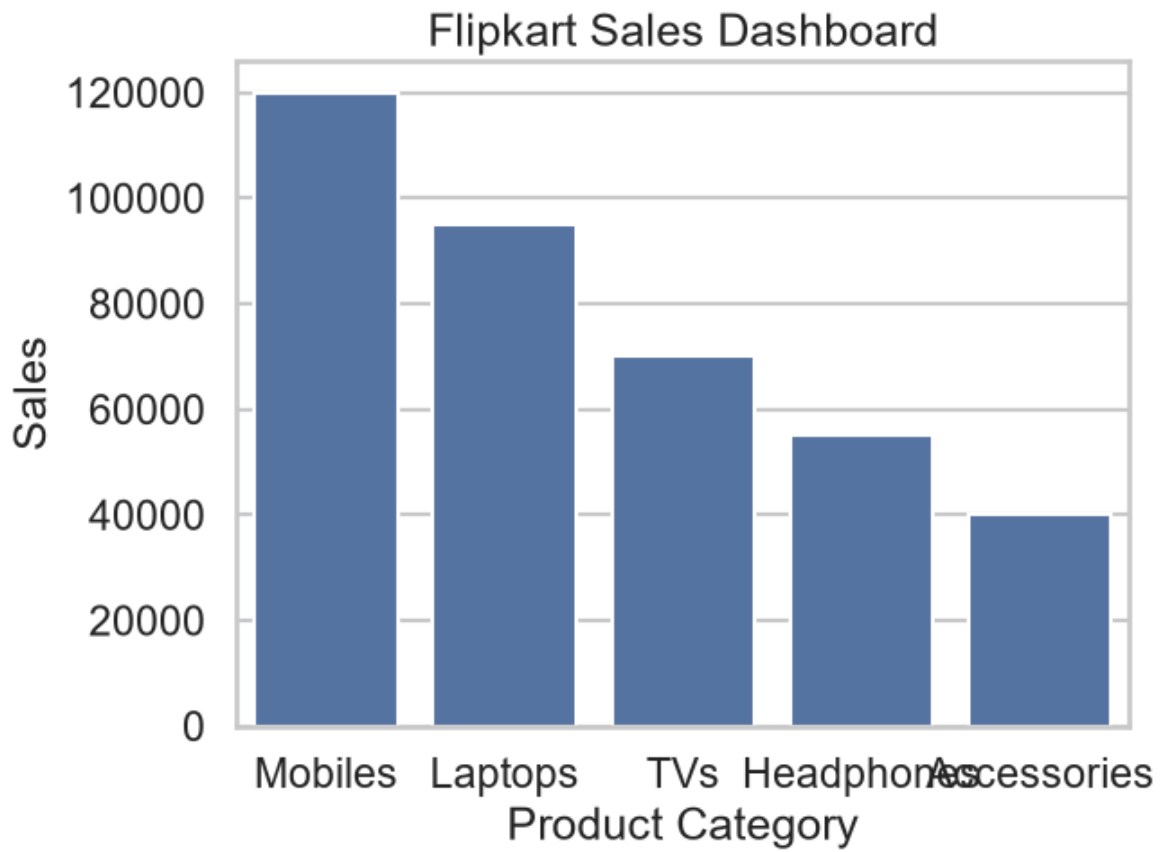
In [29]: `sns.barplot( x="category", y="sales", data=df)`

```
plt.xlabel("Product Category")
```

```
plt.ylabel("Sales")
```

```
plt.title("Flipkart Sales Dashboard")
```

Out[29]: `Text(0.5, 1.0, 'Flipkart Sales Dashboard')`



#Conclusion The Seaborn theme uses the deep palette, talk context, and white grid style to create a clean dashboard-style appearance. The barplot shows the sales of five product categories